

ROCm Migration Copilot

The AI agent that frees GPU code from CUDA lock-in — it analyzes, ports, builds, tunes, and reports. **Automatically.**

01 Analyze → 02 Port → 03 Build & Test → 04 Tune → 05 Report

THE PROBLEM

GPU code is trapped behind CUDA

Millions of lines of production GPU code — from HPC simulations to AI inference servers — compile against one vendor's proprietary stack. Moving them to AMD's open ROCm ecosystem means weeks of manual rewriting by engineers who know both platforms. Almost nobody does.

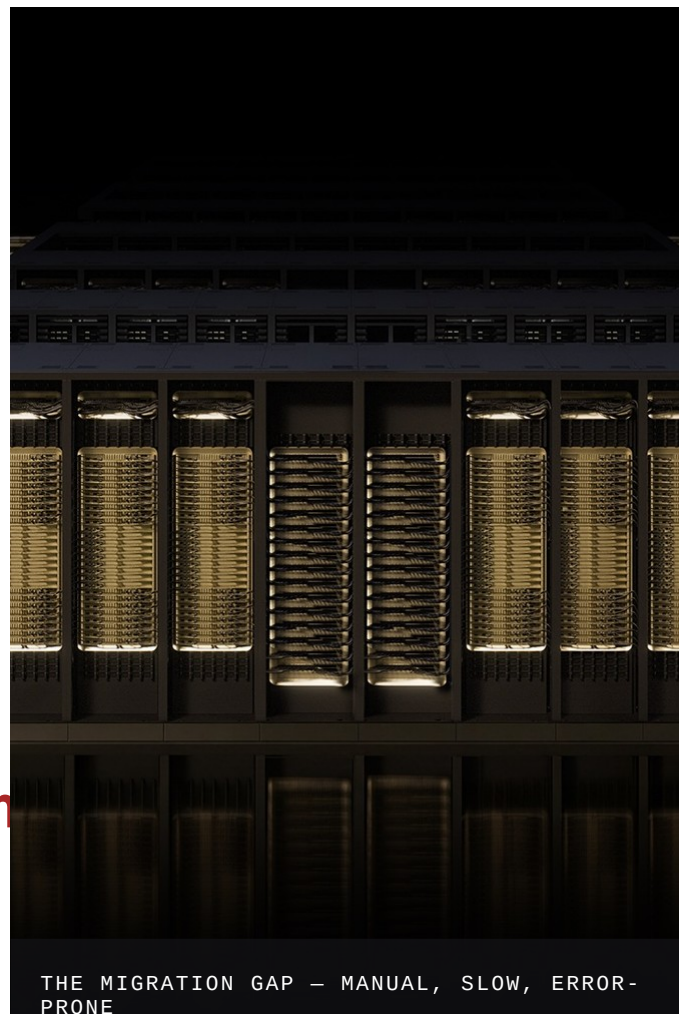
2–6 weeks 1 vendor 0 verification

typical manual port of a mid-size
repo

that every line of code
answers to

in today's syntax-only conversion
tools

hipify-perl rewrites syntax — it cannot reason about kernels, memory models, or build systems.



THE MIGRATION GAP — MANUAL, SLOW, ERROR-PRONE

From a multi-week port to an afternoon

Today — manual porting

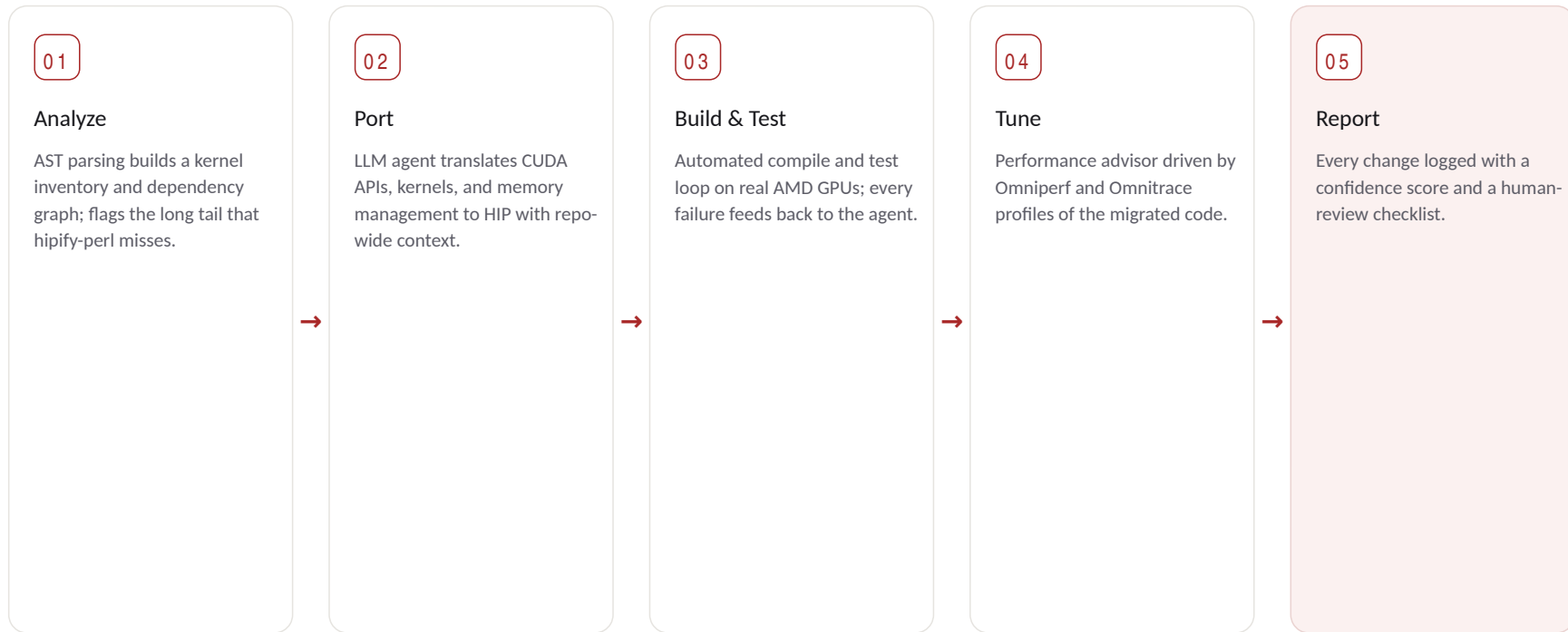
- × Grep for CUDA API calls across the repo
- × Hand-rewrite every kernel and memory path
- × Debug compiler errors one by one, for days
- × Tune blind — no AMD hardware in the loop

With ROCm Migration Copilot

- One command ingests the whole repository
- Agent plans the port before touching a line
- HIP code generated with full repo context
- Compiles, tests, and tunes on real AMD GPUs

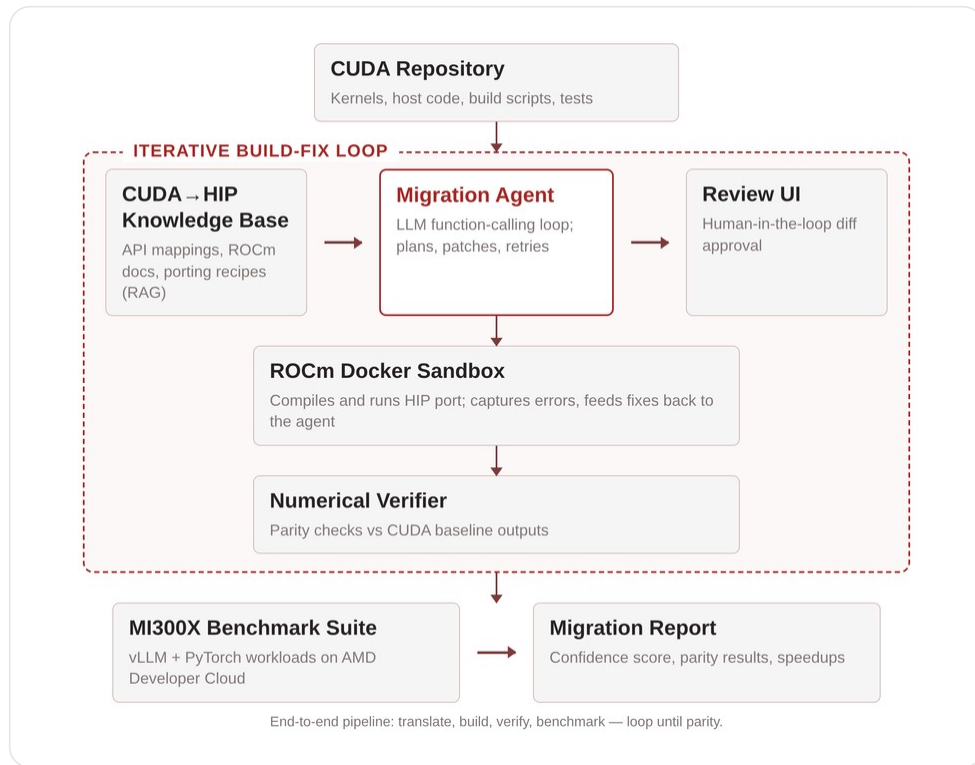
One command in → a verified, tuned, documented HIP codebase out.

A five-stage agentic pipeline



 The build-test-fix loop runs autonomously until the port compiles and passes tests.

Under the hood



Agent core

LLM with tool calling and RAG over official ROCm documentation.



AST precision

LibClang and Tree-sitter parse CUDA for surgical, structure-aware edits.



Self-healing loop

Compiler and test output drive the next fix — no human in the loop.



Orchestration

Python + LangGraph, running on ROCm-enabled MI300X cloud instances.

AMD, end to end

STACK / 01



ROCm

The open GPU software stack. Every layer inspectable, every commit public.

STACK / 02



HIP

Portable runtime and kernel language — CUDA's structure, AMD's openness.

CLOUD / 03



AMD Developer Cloud

MI300X instances through the challenge's \$100 developer credits. No hardware required.

PROFILING / 04



Omniperf + Omnitrace

First-class profiling feeds the tuning stage with real kernel-level data.

WHY IT MATTERS / 05



Runtime, cloud, profilers, docs — every layer of this project is AMD technology. **That is the point.**

WHY WE WIN

Why this beats the alternatives

STATUS QUO

hipify-perl

- × Syntax-level regex rewrites
- × Breaks on custom kernels and templates
- × No build, no test, no verification

NO VERIFICATION

THE GUESSING APPROACH

Generic AI coders

- × No GPU in the loop — errors found late
- × Hallucinate APIs without doc grounding
- × No repo-wide build context

NO GROUNDING

OUR EDGE

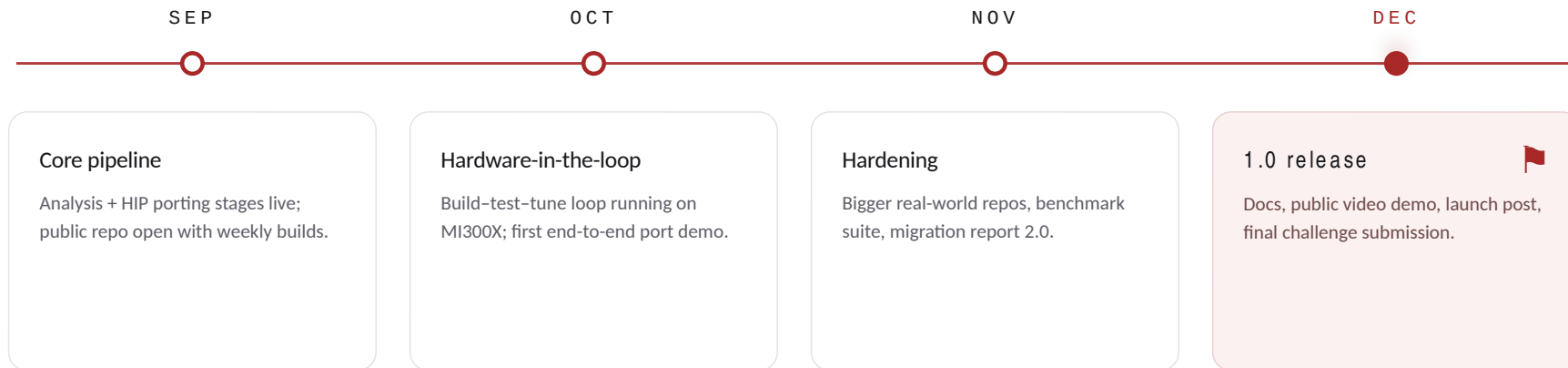
ROCm Migration Copilot

- Repo-wide reasoning before a single edit
- Real MI300X hardware verifies every change
- Confidence-scored, review-ready output

VERIFIED ON MI300X

The alternatives stop at translation — this copilot keeps building, testing, and tuning until the port is verified on real AMD hardware.

September → December 2026



SCORING

Playing the scoreboard on purpose

The challenge awards XP per category — this project is shaped to hit the highest-weight ones head-on.

+ 250

Built on AMD technology

+ 200

Complete working project

+ 200

Open source from day one

3 x 100

Monthly milestone drops

+ 150

Public demo + documentation

~ 2,475

XP PLANNED THROUGH DECEMBER

Individual scoring — one builder, full
accountability, zero coordination tax.

TEAM ROCM RANGERS

Give every CUDA codebase an escape hatch.

ROCm Migration Copilot — open source from day one, verified on MI300X.

LABLAB DISCORD — SAY HI

SEPT 1 → DEC 1 2026

BUILT ON AMD DEVELOPER CLOUD